

Progetto: Edge-REV

La nostra idea

Vorremmo proporre la nostra idea per il progetto di Sistemi Distribuiti. Vorremmo realizzare un sistema distribuito decentralizzato per l'esecuzione remota di piccoli frammenti di codice, dal dispositivo client a nodi remoti della rete.

L'obiettivo è permettere a un client di dividere carichi di lavoro a una rete di nodi worker che si autogestiscono. Il sistema si occuperà di trovare il nodo adatto, migrare il codice ed eseguirlo in isolamento, garantendo la tolleranza ai guasti tramite replicazione attiva.

1. Obiettivo:

Il nostro obiettivo con Edge-REV è mettere in pratica questi meccanismi: **naming, routing, processi e fault tolerance** — in un'unica architettura coerente. Vorremmo creare un “orchestratore” che renda l'intero processo trasparente per l'utente: il client deve quindi poter inviare uno script e ricevere il risultato senza conoscere la topologia interna della rete o il percorso fatto dai dati.

2. Architettura Proposta: Overlay P2P e Chord

L'architettura che abbiamo progettato si basa su una rete peer-to-peer pura, senza alcun coordinatore centrale. Per la gestione delle responsabilità, intendiamo implementare una **Distributed Hash Table (DHT)** basata sul modello **Chord**.

- Ogni task sarà associato a una chiave hash.
- Sfrutteremo l'anello logico di Chord per determinare quale nodo sia responsabile dell'esecuzione.
- Questo ci permetterà di avere un routing efficiente delle richieste all'interno dell'overlay.

3. Struttura sistema:

Abbiamo previsto 3 ruoli logici principali per il nostro sistema:

1. **Client:** prepara il task e interagisce con la rete tramite uno Stub locale.
2. **Peer Node:** i nodi della rete P2P che collaborano con al routing.
3. **Worker Node:** I peer che si rendono disponibili a ricevere il payload, eseguirlo in una sandbox e restituire l'output

4. Come testeremo il progetto:

Per simulare l'ambiente distribuito, intendiamo utilizzare Docker. La nostra idea prevede 3 step principali di test:

1. **Sviluppo locale** (Docker Compose): per verificare il corretto funzionamento dell'anello Chord e della finger table su una singola macchina.
2. **Deployment in Cluster:** per testare la gestione come servizi containerizzati
3. **Configurazione Multi-host:** per dimostrare il comportamento del sistema su più macchine fisiche o virtuali, simulando crash e disconnessione reali per testare la migrazione dei task.

5. Varie implementazioni (su più livelli):

Idealmente abbiamo strutturato lo sviluppo del progetto in 5 livelli incrementali:

1. Implementazione di una struttura Dispatcher/Worker sui nodi per gestire richieste multiple senza blocchi
2. Configurazione dell'overlay Chord e della finger table per l'individuazione dei responsabili.
3. Implementazione della Remote Evaluation, dove il client invia il codice come payload al Nodo responsabile.
4. Creazione di un ambiente di esecuzione protetto (una sandbox) per proteggere i nodi ospitanti.
5. Integrazione della replicazione attiva per gestire i fallimenti dei nodi durante l'elaborazione.